Day 27: Kafka & Spark Streaming

Real-time Data Processing & Fraud Detection Implementation

1. Curriculum Overview

- **Topic:** Kafka & Spark Streaming
- **Concepts Covered:** Real-time data processing, Structured Streaming, Integration patterns.

In modern data engineering, **Apache Kafka** serves as the durable, high-throughput message broker, while **Apache Spark** acts as the distributed computation engine that processes those messages on the fly.

2. Problem Statement: Real-Time Fraud Detection

We will implement a complete real-time fraud detection pipeline using your Dockerized setup (Spark 3.5.6, Python 3.12, JupyterLab 8080, Kafka 9092).

- **Producer:** Simulates financial transactions and pushes them to the `fraud-detection` Kafka topic.
- **Processing:** Spark Structured Streaming reads the transactions, filters for anomalies (e.g., amount > 10,000), enriches the stream with user details from a static PostgreSQL database (or CSV), and pushes alerts to the `fraud-notification` topic.
- **Consumer & App:** A Python script listens to `fraud-notification`. A user application allows a user to "log in" via their `userId` to receive these real-time notifications.

3. Infrastructure Setup (docker-compose.yml)

This extended Docker Compose file adds PostgreSQL to your existing Kafka and Jupyter/Spark environment.

```
version: '3.8'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:
29092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  postgres:
    image: postgres:13
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: password
      POSTGRES_DB: userdb

  jupyter-spark:
    image: jupyter/pyspark-notebook:python-3.12
    ports:
      - "8080:8888" # Jupyter mapped to 8080
    environment:
      JUPYTER_ENABLE_LAB: "yes"
    volumes:
      - ./work:/home/jovyan/work
```

4. Code Implementation

4.1 User Database / CSV (user_table.csv)

We use this static data to enrich our streaming transactions. In production, this would reside in PostgreSQL.

userId	name	email	account_status
101	Alice Smith	alice@example.com	active
102	Bob Jones	bob@example.com	active

4.2 The Producer (producer.ipynb)

Run this in JupyterLab to generate transactions.

```
from kafka import KafkaProducer
import json
import time
import random

producer = KafkaProducer(
    bootstrap_servers=['kafka:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

user_ids = [101, 102, 103, 104, 105]

print("Starting Transaction Producer...")
for i in range(100):
    tx_data = {
        "tx_id": i,
        "userId": random.choice(user_ids),
        "amount": random.uniform(10.0, 15000.0), # Some amounts > 10000 will trigger
fraud
        "timestamp": time.time()
    }
    producer.send('fraud-detection', value=tx_data)
    print(f"Sent: {tx_data}")
    time.sleep(2)
```

4.3 Spark Stream Processing (fraud_processing.ipynb)

This script reads the Kafka stream, detects fraud, enriches it with User DB data, and pushes it to a new Kafka topic.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col, struct, to_json
from pyspark.sql.types import StructType, StructField, IntegerType, DoubleType

# Initialize Spark with Kafka and Postgres packages
spark = SparkSession.builder \
    .appName("FraudDetection") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-
kafka-0-10_2.12:3.5.3,org.postgresql:postgresql:42.6.0") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

# 1. Load Static User Data (From CSV or Postgres)
# If using CSV: users_df = spark.read.csv("user_table.csv", header=True,
inferSchema=True)
users_df = spark.read \
    .format("jdbc") \
    .option("url", "jdbc:postgresql://postgres:5432/userdb") \
    .option("dbtable", "users") \
    .option("user", "admin") \
    .option("password", "password") \
    .load()

# 2. Read Streaming Data from Kafka
tx_schema = StructType([
    StructField("tx_id", IntegerType(), True),
    StructField("userId", IntegerType(), True),
    StructField("amount", DoubleType(), True),
    StructField("timestamp", DoubleType(), True)
])

kafka_stream = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "fraud-detection") \
    .load()

# 3. Parse and Filter Data
parsed_stream = kafka_stream.select(from_json(col("value").cast("string"),
tx_schema).alias("tx")).select("tx.*")

# Fraud Logic: Amount > 10,000
fraud_stream = parsed_stream.filter(col("amount") > 10000.0)

# 4. Enrich Stream with User Details (Stream-Static Join)
enriched_fraud = fraud_stream.join(users_df, "userId")
```

```
# 5. Format for output Kafka topic
output_stream = enriched_fraud \
    .withColumn("value", to_json(struct("*")).cast("string")) \
    .select("value")

# 6. Write Stream to 'fraud-notification' Topic
query = output_stream.writeStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("topic", "fraud-notification") \
    .option("checkpointLocation", "/tmp/checkpoints") \
    .start()

query.awaitTermination()
```

4.4 User App & Consumer (user_app.py)

A client-side application where a user logs in (via ID) and listens for their specific notifications.

```
from kafka import KafkaConsumer
import json

def user_login_and_listen():
    print("=== Fraud Alert System ===")
    user_id_input = input("Enter your userId to login (No password required): ")

    try:
        user_id = int(user_id_input)
    except ValueError:
        print("Invalid ID. Exiting.")
        return

    print(f"Logged in successfully as User {user_id}. Listening for real-time
alerts...")

    # Initialize Kafka Consumer listening to the notification topic
    consumer = KafkaConsumer(
        'fraud-notification',
        bootstrap_servers=['localhost:9092'], # Adjust to 'kafka:9092' if running
inside docker
        auto_offset_reset='latest',
        value_deserializer=lambda x: json.loads(x.decode('utf-8'))
    )

    for message in consumer:
        alert_data = message.value

        # Only show alerts meant for the logged-in user
        if alert_data.get('userId') == user_id:
            print("\n[CRITICAL ALERT] 🚨")
            print(f"User Name: {alert_data.get('name')}")
            print(f"Suspicious Transaction ID: {alert_data.get('tx_id')}")
            print(f"Amount: ${alert_data.get('amount'):.2f}")
            print("Please verify this transaction immediately.\n")

if __name__ == "__main__":
    user_login_and_listen()
```